진행 보고 · 2026-10-15 · 시뮬레이션(Gazebo) 단계

말로 시키면 블록을 옮기는 로봇 팔

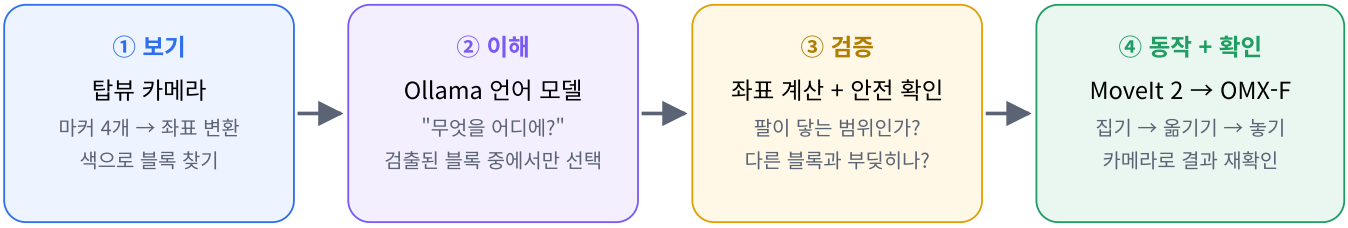
ROBOTIS OMX-F · ROS 2 Jazzy · MoveIt 2 · Ollama(gemma4:e2b) · 탑뷰 카메라 + ArUco 마커 4개

"빨간 블록을 파란 블록 위에 쌓아줘" 같은 한국어/영어 명령을 받아, 카메라로 블록을 찾고, 언어 모델이 할 일을 정하고, 로봇 팔이 실제로 집어서 옮깁니다. Depth 카메라 없이 **테이블 네 꼭짓점의 ArUco 마커**만으로 블록의 실제 위치(m)를 계산합니다.

5 / 5	2.1 mm	10 / 10	≈ 1–2 s
최종 자연어 명령 테스트 (실행 4 + 올바른 거부 1)	카메라로 구한 블록 위치 최대 오차 (실제 위치 대비)	언어 모델 명령 해석 정확도 (개선 전 8/10)	명령 해석 시간 (첫 호출만 ≈15 s, 모델 로딩)

어떻게 동작하나요?

사용자: "파란 블록을 빨간 블록 위에 쌓아줘"



핵심 설계: 언어 모델은 **좌표를 만들지 않고** 블록 이름과 놓을 장소만 고릅니다. 좌표는 코드가 계산하므로 작은 모델이 틀린 숫자를 지어낼 수 없습니다.

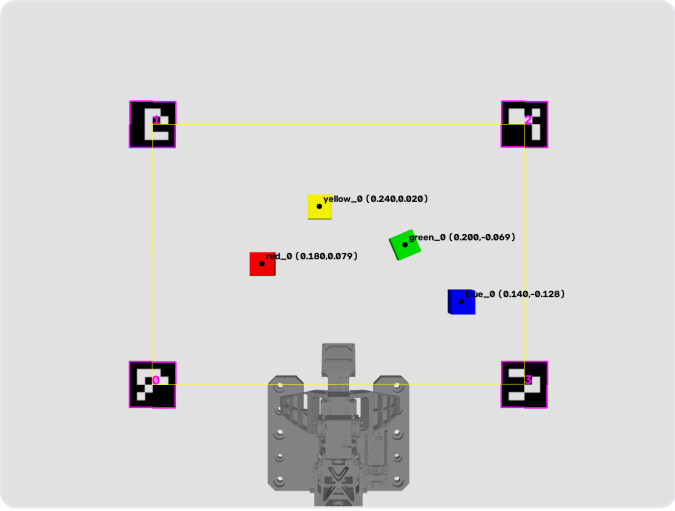
지금 할 수 있는 명령

영역으로 옮기기	"빨간 블록을 **가운데**로"
다른 블록 옆에	"초록 블록을 노란 블록 **뒤에**"
위에 쌓기	"파란 블록을 빨간 블록 **위에**"
여러 단계	"A를 B 위에 올리고, C를 A 위에"
불가능한 요청	"보라색 블록…" → 이유와 함께 거부

현재 단계

설치·빌드	완료
시뮬레이션 환경	완료
카메라 인식	완료
로봇 동작	완료
손목 카메라 집기 확인	완료
캘리브레이션 도구	완료
언어 명령 해석	완료
실물 로봇 이전	다음 단계

① 보기 — Depth 없이 마커 4개로 위치 재기

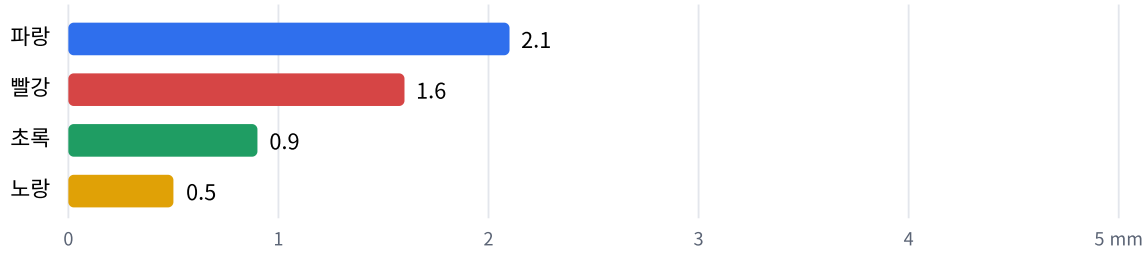


카메라 화면 위에 인식 결과를 표시한 모습. 분홍 테두리 = ArUco 마커(0~3), 검은 점 = 블록 중심, 괄호 = 로봇 기준 실제 좌표(m).

1. **마커 4개를 찾는다**
   네 꼭짓점 마커의 실제 위치(로봇 기준 m)를 미리 알고 있습니다.
2. **화면 ↔ 테이블 변환식을 만든다**
   마커 4점으로 "화면의 점 → 테이블 위 실제 위치" 변환(호모그래피)을 계산합니다. 카메라 높이도 함께 역산됩니다(0.75 m, 실제와 일치).
3. **색으로 블록을 찾는다**
   빨강·초록·파랑·노랑 영역을 찾고, 블록이 회전한 각도도 구합니다.
4. **블록 높이만큼 보정한다**
   카메라는 블록 윗면을 보므로, 원근 때문에 생기는 위치 밀림(최대 ≈7 mm)을 카메라 위치를 이용해 보정합니다.

정확도 — 4개 블록 모두 시뮬레이터의 실제 위치와 비교해 오차 **0.5 ~ 2.1 mm**, 회전 각도 오차 **0°**.

블록별 위치 오차 (카메라 추정 vs 실제)



② 이해 — 언어 모델이 "무엇을 어디에"를 고른다

언어 모델에게 주는 것

```
블록 목록:
red_0 (0.180, 0.079)
green_0 (0.200, -0.069)
blue_0 (0.140, -0.128)
yellow_0 (0.240, 0.020)

명령: 파란 블록을 빨간 블록 위에 쌓아줘
```

언어 모델이 돌려주는 것

```
{ "feasible": true,
  "steps": [
    { "pick": "blue_0",
      "place": "on_top:red_0" } ] }
```

→ 코드가 red_0의 위치와 높이(+2.5 cm)로 좌표를 계산

개선 전후 비교 (명령 10개, gemma4:e2b)



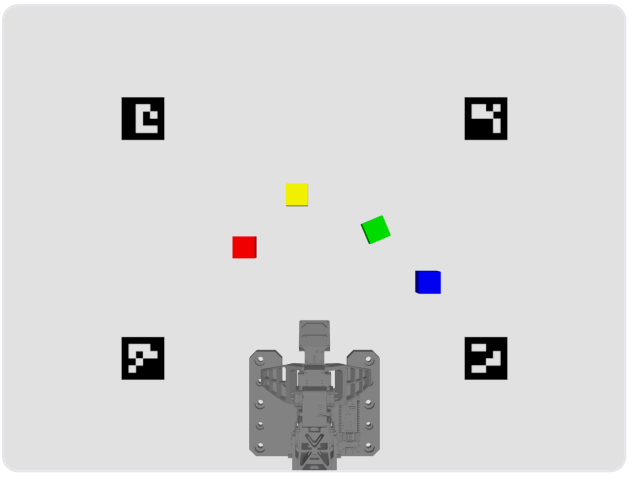
평균 시간에는 첫 호출의 모델 로딩(≈15 s)이 포함됨. 이후 호출은 1~2 s.

무엇을 바꿨나? 처음에는 모델이 블록 이름 자리에 "id"라는 글자를 그대로 쓰거나, 장소 이름을 빠뜨리는 실수가 있었습니다. 지금은 **실제로 보이는 블록 이름과 허용된 장소 목록 안에서만 고를 수 있도록** 출력 형식을 매번 새로 만들어 강제합니다. 작은 모델(2B급)로도 실수가 사라졌습니다.

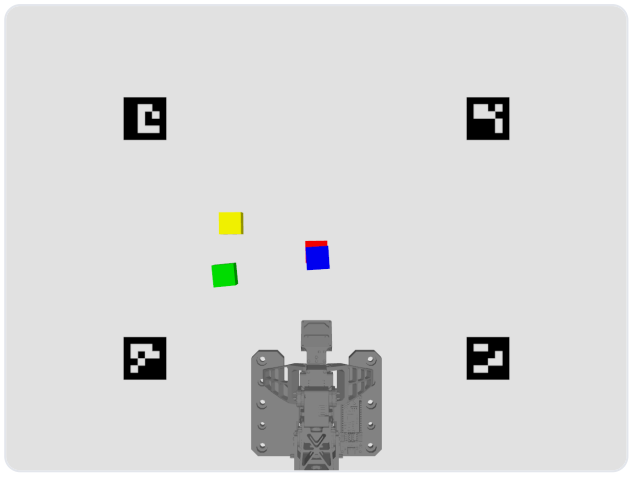
검증 단계에서 막아주는 것

상황	결과
없는 블록을 지시 ("보라색 블록")	언어 모델이 거부 + 이유 설명
팔이 닿지 않는 곳 (반경 25 cm 밖)	움직이기 전에 거부
놓을 자리에 다른 블록이 있음 (3.5 cm 이내)	움직이기 전에 거부
위에 다른 블록이 쌓인 블록을 집으려 함	움직이기 전에 거부
옮긴 뒤 블록이 목표에서 3 cm 이상 벗어남 / 쌓기가 무너짐	카메라로 확인해 실패로 보고

④ 동작 — 실제로 옮긴 결과



시작 상태



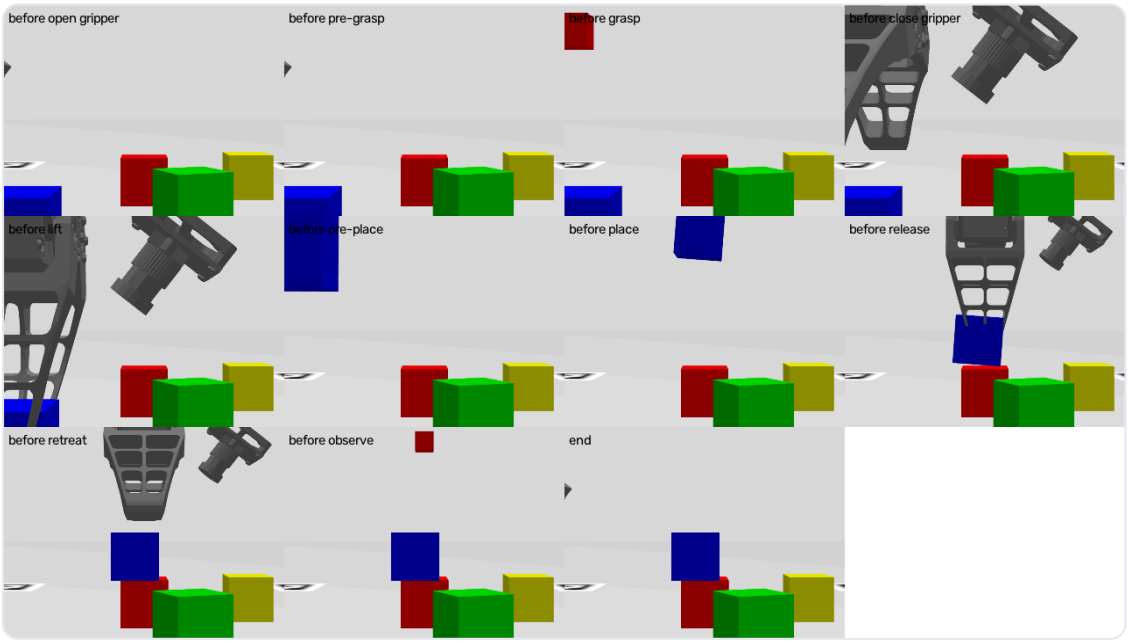
명령 4개 실행 후 (빨강 → 가운데, 파랑 → 빨강 위, 노랑 → 왼쪽 앞, 초록 → 노랑 뒤)

최종 테스트 결과

명령	결과	목표와의 거리	걸린 시간
빨간 블록을 가운데로 옮겨	성공	5.2 mm	32 s*
파란 블록을 빨간 블록 위에 쌓아줘	성공 (쌓임 확인)	6.1 mm	18 s
노란 블록을 왼쪽 앞으로 옮겨	성공	3.5 mm	19 s
초록 블록(회전된 상태)을 노란 블록 뒤에 놔	성공	12.6 mm	21 s
보라색 블록을 가운데로 옮겨	올바르게 거부	–	1 s

* 첫 명령은 언어 모델 로딩 15 s 포함. 블록 하나 옮기는 동작 자체는 약 15–18 s.

쌓기 동작을 옆에서 본 모습

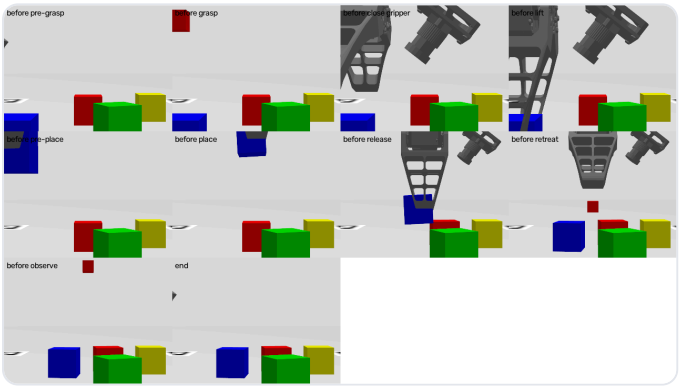


열기 → 접근 → 내려가기 → 집기 → 들기 → 이동 → 내려놓기 → 놓기 → 복귀. 파란 블록이 빨간 블록 위에 올라갔습니다.

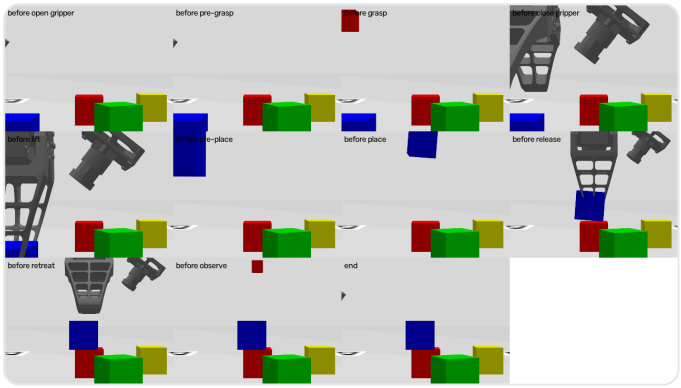
개발 중 해결한 문제들

처음 통합 실행에서는 "성공"이라고 보고했지만 블록이 옮겨지지 않았습니다. 원인을 하나씩 찾아 고쳤습니다.

증상	원인	해결
집게가 블록 위에서 멈추고 블록을 끝부분만 잡음	시뮬레이터의 관절 위치 제어 힘(게인)이 너무 약해 팔이 중력에 처짐 (최대 0.04 rad ≈ 2°)	게인 0.1 → 1.0. 관절 오차 **0**으로
다음 동작이 거부됨	팔이 완전히 멈추기 전에 다음 경로를 보냄	관절 속도가 0이 될 때까지 기다린 뒤 실행
집게를 끝까지 닫으면 블록이 튀어나감	닫힘 목표가 블록 폭보다 훨씬 작아 과도하게 조임	닫힘 값을 블록 크기에 맞게 조정
쌓을 때 아래 블록을 밀어버림	이동 중 블록이 집게 안에서 기울어 2 cm 어긋남	잡는 힘을 높여 기울어짐 방지
실패했는데 "성공"으로 보고	동작 결과를 확인하지 않음	놓은 뒤 카메라로 위치·쌓임 확인
언어 모델이 "id" 같은 가짜 이름 출력	출력 형식 제약이 약함	보이는 블록 이름만 고르도록 강제 (8/10 → 10/10)



수정 전: 약하게 잡은 파란 블록이 기울어져 빨간 블록 옆에 떨어짐



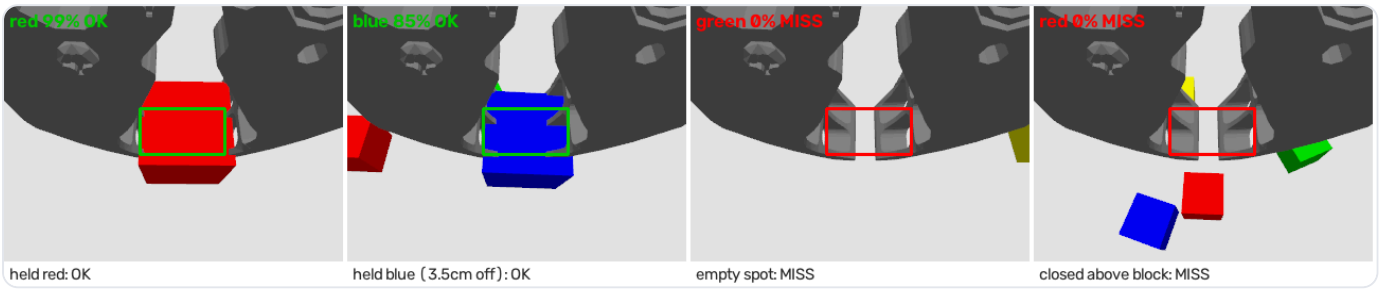
수정 후: 블록이 집게 중심에 고정되어 정확히 쌓임

반복 시험 (각 3회)

시험	성공	비고
일반 이동	**3 / 3**	목표와 4.3 ~ 5.4 mm, 매번 같은 방향으로 어긋남 → 보정 가능
회전된 블록(23°) 집기	**3 / 3**	집게 방향을 블록 각도에 맞춰 회전
쌓기	**2** / 3 → **8 / 9**	놓기 방식 개선 후 (다음 쪽 참고)

손목 카메라로 "제대로 잡았나?" 확인

블록을 들어 올린 직후 **그리퍼에 달린 손목 카메라**로 손가락 사이를 봅니다. 잡은 블록은 카메라 바로 앞에 있어 화면 가운데 상자를 거의 가득 채우고, 놓쳤다면 상자가 비어 있습니다. 상자 안에서 **집으려던 색이 60% 이상**이면 통과, 아니면 집게를 열고 물러난 뒤 실패로 보고합니다.



손목 카메라 화면. 초록 상자 = 통과, 빨간 상자 = 실패. 왼쪽부터 정상 집기 · 3.5 cm 어긋났지만 끌려 들어와 잡힘 · 빈 자리를 집음 · 블록 위에서 헛잡음(블록은 아래 테이블에 남음).

시험 결과

상황	색 비율	판정
정상 집기 (반복 8회)	85–99%	통과
빈 자리를 집음	0%	실패
블록 위에서 헛잡음	0%	실패
다른 색 블록을 집음	0%	실패

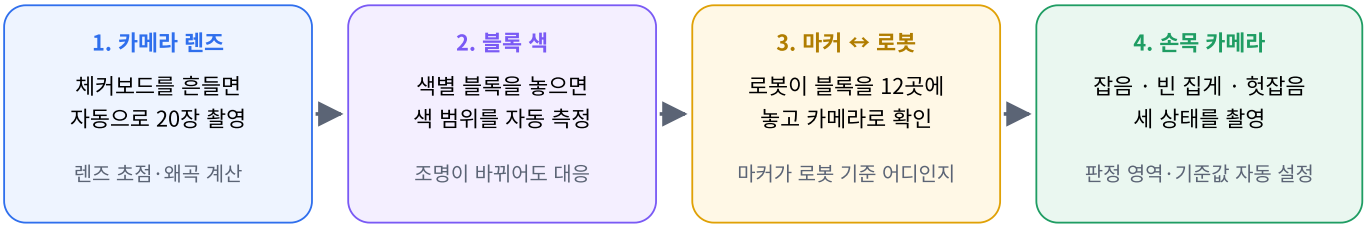
함께 개선된 점

- 쌓을 때 4 mm 위에서 놓기 → 아래 블록이 튕겨 나가지 않음
- 놓을 때 집게를 조금만 열기 → 블록이 날아가지 않음
- 쌓기 성공률 **2/3 → 8/9**
- 일반 이동 3/3 유지 (오차 4–6 mm)

남은 과제 — 위에서 내려다보는 카메라로는 쌓은 블록 아래가 "가려진 것"인지 "튕겨 나간 것"인지 구분하지 못합니다. 놓은 뒤 손목 카메라로 위 블록의 크기를 재서 실제로 쌓였는지 확인하는 기능을 다음에 추가할 계획입니다.

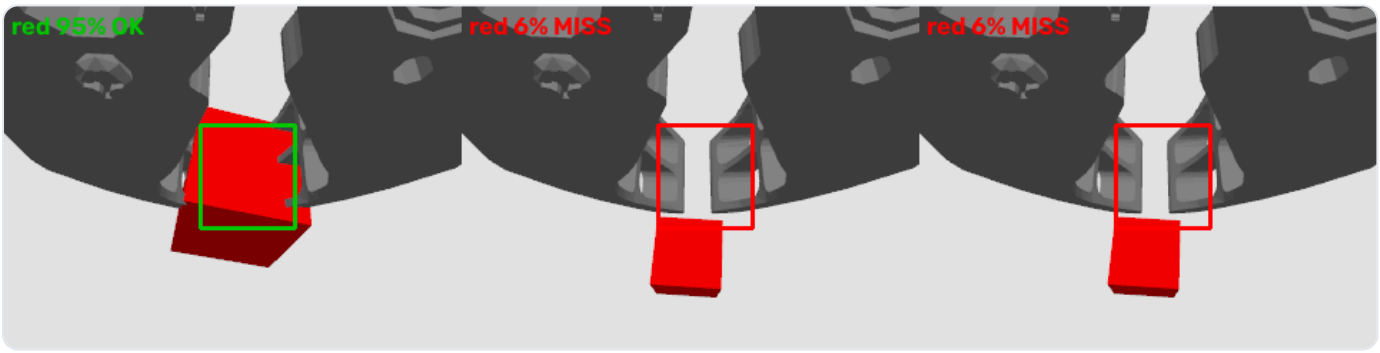
캘리브레이션 — 실제 환경에 맞추기

카메라, 마커 위치, 조명, 손목 카메라는 설치할 때마다 달라집니다. 이를 맞추는 **4단계 도구**를 만들었고, 시뮬에서 정답과 비교해 검증했습니다. 결과는 설정 파일로 저장되고, 실행할 때 `calib_dir`로 지정하면 자동으로 적용됩니다.



시뮬 검증 결과

단계	검증 방법	결과
1. 카메라 렌즈	시뮬 카메라의 실제 값과 비교	**초점거리 오차 0.03 %**
2. 블록 색	4색 블록 측정	**4색 모두 인식, 98–100 %, 색끼리 겹침 없음**
3. 마커 ↔ 로봇	마커 위치를 일부러 2° 돌리고 1.4 cm 옮긴 뒤 실행	**회전 2.04° 되찾음, 맞춤 오차 3 mm**
4. 손목 카메라	잡음과 놓침 구분	**잡음 95–100 % vs 놓침 6 % → 확실히 구분**



손목 카메라 캘리브레이션이 자동으로 찍은 세 장면(잡음 · 빈 집게 · 헛잡음)과 정해진 판정 영역

알아둘 점 — 3단계는 카메라 좌표를 **"로봇이 실제로 블록을 놓는 위치"**에 맞춥니다. 그래서 카메라와 로봇 사이의 일치는 0–1 mm로 좋아지지만, 시뮬에서는 로봇이 늘 약 9 mm 짧게 놓는 버릇까지 함께 반영되어 절대 위치는 그만큼 이동합니다. 블록 위에 쌓기·옆에 놓기처럼 블록 기준 작업에 유리한 방식입니다.

다음 단계

시뮬레이션에서 더 할 일

- 놓는 위치의 일정한 어긋남(≈4 mm) 보정
- 쌓기 성공률 높이기 (내려놓는 속도 낮추기 등)
- 명령 시나리오를 늘려 성공률 통계 측정
- 더 큰 언어 모델과 비교 (16 GB GPU 안에서)

실물 로봇으로 옮길 때 필요한 것

- **마커**: 마커 사이 거리만 자로 재기 (로봇 기준 위치는 캘리브레이션 도구가 구함)
- **카메라**: 장치 번호, 해상도 → 렌즈 캘리브레이션 도구
- **블록**: 실제 크기, 색 → 색 캘리브레이션 도구
- **로봇**: U2D2 포트, 그리퍼 잡는 힘 확인
- 원하는 명령 예시 5~10개

실물 이전이 쉬운 이유 — 카메라 인식, 언어 모델, 검증 로직은 시뮬레이션과 실물에서 그대로 쓰입니다. 바뀌는 것은 **설정 파일 값**(마커 좌표, 색 범위, 잡는 높이)과 **카메라·로봇 드라이버 실행**뿐입니다.

실행 방법 (요약)

```
# 터미널 1 — 전체 실행 (Gazebo + RViz + 인식 + 언어 모델 + 동작)
ros2 launch vla_bringup sim_vla.launch.py

# 터미널 2 — 한국어로 명령
ros2 run vla_planner cli
> 빨간 블록을 가운데로 옮겨
```

자세한 내용은 기술 문서(report_technical.pdf)와 README.md 참고.